

04 · SELECT — Querying and Filtering

Why this matters in interviews. This is the chapter that decides most SQL screening rounds. Expect to *write* queries live: filter with `AND/OR` precedence, `LIKE` patterns, `IN / BETWEEN`, sort with `ORDER BY`, get the *top N*, and explain the **order in which a query executes**.

In this chapter: `SELECT` and `DISTINCT` · the logical order of execution · `WHERE` and its operators · `AND / OR / NOT` and precedence · `IN`, `BETWEEN`, `LIKE` and wildcards · `IS NULL` · `ORDER BY` · `CASE` · limiting rows (top-N, `FETCH FIRST`, `ROWNUM`) · aliases.

All examples use the sample tables from **00 · Start Here**. The two you will meet most:

dept

dept_id	dept_name	location
10	Management	Chennai
20	Engineering	Bengaluru
30	Sales	Mumbai
40	HR	Pune

emp

emp_id	emp_name	job	manager_id	salary	comm	dept_id	hire_date
101	Asha	CEO	NULL	90000	NULL	10	2015-01-10
102	Bala	Manager	101	70000	NULL	20	2016-03-15
103	Chitra	Developer	102	50000	NULL	20	2018-07-01
104	Dinesh	Developer	102	50000	NULL	20	2019-02-20
105	Esha	Tester	102	40000	NULL	20	2020-11-05
106	Farhan	Manager	101	65000	NULL	30	2017-05-30
107	Gita	Sales Rep	106	35000	5000	30	2021-01-12
108	Hari	Sales Rep	106	35000	3000	30	2022-06-18
109	Isha	Intern	103	20000	NULL	20	2024-01-08
110	Jai	Consultant	101	45000	NULL	NULL	2023-04-01

4.1 SELECT — choosing columns

`SELECT` is used to **select data from a database**. The result is a *result set* (a temporary table).

```
SELECT column1, column2, ...  
FROM table_name;
```

`column1, column2, ...` are the **field names** of the table you want to select from. `SELECT *` returns **all** columns.

```
SELECT emp_name, job FROM emp;
```

Result

emp_name	job
Asha	CEO
Bala	Manager
Chitra	Developer
Dinesh	Developer
Esha	Tester
Farhan	Manager
Gita	Sales Rep
Hari	Sales Rep
Isha	Intern
Jai	Consultant

Style rule. Use `SELECT *` only for a quick look. In real code (and in interviews) name the columns — it is faster, clearer and won't break when a column is added.

`SELECT DISTINCT` — remove duplicates

Inside a table a column often contains many duplicate values, and sometimes you only want the **different (distinct) values**. `SELECT DISTINCT` returns only those.

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

```
SELECT DISTINCT job FROM emp;
```

Result

job
CEO
Manager
Developer
Tester
Sales Rep
Intern
Consultant

`DISTINCT` applies to the **whole row** you select, not just the first column, and all NULLs count as one value:

```
SELECT DISTINCT dept_id FROM emp ORDER BY dept_id;
```

Result

dept_id
10
20
30
NULL

```
SELECT DISTINCT dept_id, job FROM emp WHERE dept_id = 20 ORDER BY job;
```

Result

dept_id	job
20	Developer
20	Intern
20	Manager
20	Tester

Two `Developer` rows in department 20 collapse into one because the *pair* `(20, Developer)` is identical.

Expressions and concatenation

You may select any expression, not just columns:

```
SELECT emp_name, salary, salary * 12 AS annual_pay, emp_name || ' (' || job || ')' AS label
FROM emp WHERE dept_id = 30;
```

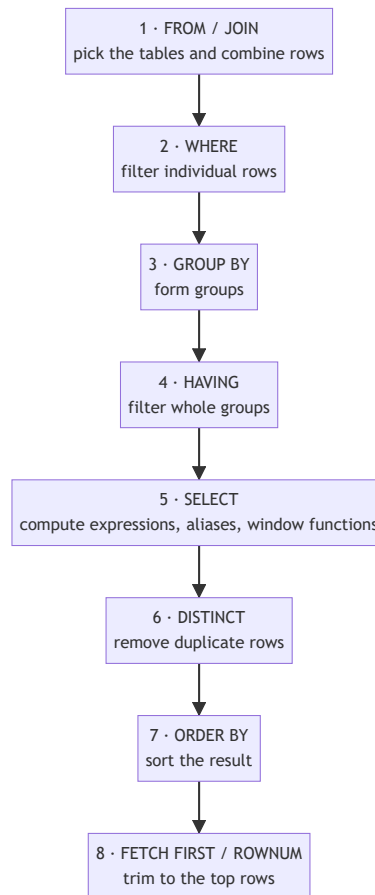
Result

emp_name	salary	annual_pay	label
Farhan	65000	780000	Farhan (Manager)
Gita	35000	420000	Gita (Sales Rep)
Hari	35000	420000	Hari (Sales Rep)

`||` is Oracle's string-concatenation operator (Chapter 01).

4.2 The logical order of execution

You **write** a query starting with `SELECT`, but the database **evaluates** it in a different order. Knowing this explains a whole family of "why does this fail?" questions.



Consequences you can quote in an interview:

Fact	Because ...
A column alias cannot be used in <code>WHERE</code> (nor in <code>GROUP BY</code> / <code>HAVING</code> in Oracle up to 21c)	aliases are created in step 5, after those clauses ran
An alias can be used in <code>ORDER BY</code>	sorting happens after <code>SELECT</code>
<code>WHERE</code> cannot contain <code>SUM()</code> , <code>COUNT()</code> ...	groups do not exist yet — that is the job of <code>HAVING</code> (Chapter 06)
Window functions (Chapter 11) cannot appear in <code>WHERE</code>	they run in step 5 — filter on them with a subquery
<code>ROWNUM</code> is assigned before <code>ORDER BY</code>	see §4.10

4.3 `WHERE` — filtering rows

The `WHERE` clause **filters records**: it extracts only those rows that fulfil a specified condition.

```
SELECT column1, column2, ...
FROM   table_name
WHERE  condition;
```

```
SELECT emp_name, job, salary FROM emp WHERE dept_id = 30;
```

Result

emp_name	job	salary
Farhan	Manager	65000
Gita	Sales Rep	35000
Hari	Sales Rep	35000

Text vs numeric values

SQL requires **single quotes around text values**; **numeric values are not quoted**.

```
SELECT * FROM emp WHERE job = 'Manager';      -- text → quoted
SELECT * FROM emp WHERE emp_id = 102;         -- number → no quotes
SELECT * FROM emp WHERE salary > 50000;
```

(Most databases also accept double quotes for text; **in Oracle double quotes mean an identifier**, so always use single quotes.) Oracle string comparison is **case-sensitive**: `job = 'manager'` matches nothing.

Operators you can use in WHERE

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal (in some versions of SQL this may be written !=)
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	Specify multiple possible values for a column

4.4 AND, OR, NOT — and the precedence trap

AND — all conditions must be TRUE (a **WHERE** can contain one or many **AND** s):

```
SELECT emp_name, job, salary FROM emp WHERE dept_id = 20 AND salary >= 50000;
```

Result

emp_name	job	salary
Bala	Manager	70000
Chitra	Developer	50000
Dinesh	Developer	50000

```
SELECT column1, column2, ...  
FROM   table_name  
WHERE  condition1 AND condition2 AND condition3 ...;
```

OR — at least one condition must be TRUE:

```
SELECT emp_name, dept_id FROM emp WHERE dept_id = 10 OR dept_id = 30;
```

Result

emp_name	dept_id
Asha	10
Farhan	30
Gita	30
Hari	30

NOT — gives the opposite ("negative") result and can be combined with other operators:

```
SELECT emp_name, dept_id FROM emp WHERE NOT dept_id = 20;
```

Result

emp_name	dept_id
Asha	10
Farhan	30
Gita	30
Hari	30

```
SELECT column1, column2, ... FROM table_name WHERE NOT condition;
```

Notice **Jai (dept_id is NULL) is missing** from that result: `NOT (NULL = 20)` is `NOT UNKNOWN = UNKNOWN`, so the row is dropped. To include him: `WHERE dept_id <> 20 OR dept_id IS NULL`.

Precedence: `NOT` → `AND` → `OR`

Operator	Precedence	Description
<code>NOT</code>	Highest	negates a condition
<code>AND</code>	Medium	evaluated before <code>OR</code>
<code>OR</code>	Lowest	evaluated last
<code>()</code>	Manual	overrides the default order

You can combine `AND` and `OR` — **use parentheses** to get the result you mean. *Goal: managers or the CEO — but only those who work in department 30 (Sales).*

With parentheses (correct):

```
SELECT emp_name, job, dept_id FROM emp
WHERE dept_id = 30 AND (job = 'Manager' OR job = 'CEO');
```

Result

emp_name	job	dept_id
Farhan	Manager	30

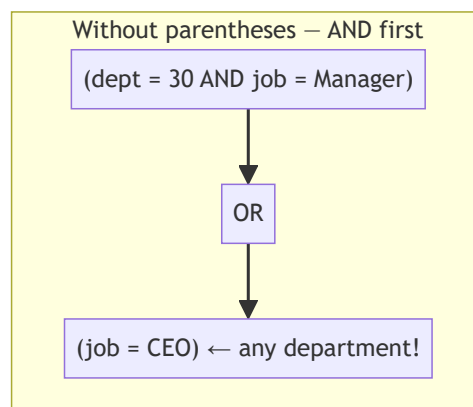
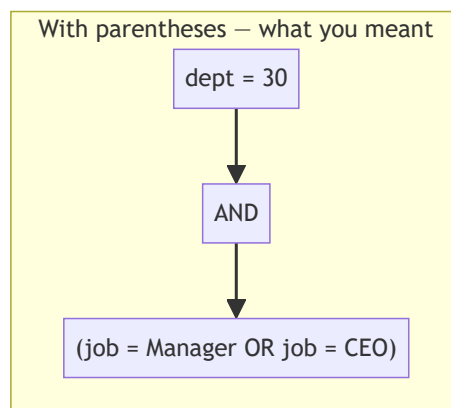
Without parentheses `AND` binds first, so the database reads it as `(dept_id = 30 AND job = 'Manager')` OR `job = 'CEO'` — and the CEO from a *different* department slips in:

```
SELECT emp_name, job, dept_id FROM emp
WHERE dept_id = 30 AND job = 'Manager' OR job = 'CEO';
```

Result

emp_name	job	dept_id
Asha	CEO	10
Farhan	Manager	30

Same words, different meaning: **Asha (CEO, department 10) appears** in the second result. The classic textbook version of this trap: "all customers from Spain that start with G plus all customers that start with R, regardless of country". **Always parenthesise mixed AND / OR**.



4.5 IN, BETWEEN

IN — a list of values

IN lets you specify multiple values in a WHERE clause. It is **shorthand for multiple OR conditions**.

```
SELECT column_name(s) FROM table_name WHERE column_name IN (value1, value2, ...);
```

```
SELECT emp_name, job FROM emp WHERE job IN ('Manager', 'CEO', 'Intern');
```

Result

emp_name	job
Asha	CEO
Bala	Manager
Farhan	Manager
Isha	Intern

NOT IN returns rows that are **not** any of the listed values:

```
SELECT emp_name, job FROM emp WHERE job NOT IN ('Developer', 'Tester', 'Sales Rep');
```

Result

emp_name	job
Asha	CEO
Bala	Manager
Farhan	Manager
Isha	Intern
Jai	Consultant

IN also accepts a **subquery** — *"return all customers that have an order in the Orders table"*: WHERE CustomerID IN (SELECT CustomerID FROM Orders). And NOT IN (SELECT ...) finds those with **no** order. (Deep dive, including the NULL trap, in Chapter 08.)

BETWEEN — a range, inclusive

BETWEEN selects values **within a given range**. The values can be **numbers, text or dates**, and the operator is **inclusive**: both the begin and the end value are included.

```
SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2;
```

```
SELECT emp_name, salary FROM emp WHERE salary BETWEEN 40000 AND 50000 ORDER BY salary, emp_name;
```

Result

emp_name	salary
Esha	40000
Jai	45000
Chitra	50000
Dinesh	50000

x BETWEEN a AND b is exactly $x \geq a$ AND $x \leq b$ — write the **smaller value first** (BETWEEN 50000 AND 40000 returns nothing). NOT BETWEEN selects everything outside the range:

```
SELECT emp_name, salary FROM emp WHERE salary NOT BETWEEN 30000 AND 70000 ORDER BY salary;
```

Result

emp_name	salary
Isha	20000
Asha	90000

Text values — alphabetical order:

```
SELECT emp_name FROM emp WHERE emp_name BETWEEN 'Bala' AND 'Esha' ORDER BY emp_name;
```

Result

emp_name
Bala
Chitra
Dinesh
Esha

Dates — use a real date literal (or TO_DATE), never a loose string:

```
SELECT emp_name, hire_date FROM emp
WHERE hire_date BETWEEN DATE '2018-01-01' AND DATE '2020-12-31'
ORDER BY hire_date;
```

Result

emp_name	hire_date
Chitra	2018-07-01
Dinesh	2019-02-20
Esha	2020-11-05

Dates in the source. W3Schools writes `BETWEEN #07/01/1996# AND #07/31/1996#` — the `#...#` delimiters are **MS Access** only. The portable forms are `'1996-07-01'` (MySQL), and in Oracle `DATE '1996-07-01'` or `TO_DATE('01-JUL-1996', 'DD-MON-YYYY')`.

Trap — DATE columns carry a time. In Oracle a `DATE` includes hours/minutes/seconds. `order_date BETWEEN DATE '1996-07-01' AND DATE '1996-07-31'` **misses everything on 31 July after midnight**. Safer: `order_date >= DATE '1996-07-01' AND order_date < DATE '1996-08-01'`.

4.6 LIKE and wildcards

`LIKE` searches for a **specified pattern** in a column inside a `WHERE` clause. Two wildcards are used with it:

- `%` — represents **zero, one or many** characters
- `_` — represents **exactly one** character

```
SELECT column1, column2, ... FROM table_name WHERE columnN LIKE pattern;
```

Goal	Pattern	Meaning
Starts with A	<code>LIKE 'A%'</code>	put <code>%</code> at the end
Ends with a	<code>LIKE '%a'</code>	put <code>%</code> at the beginning
Contains ha	<code>LIKE '%ha%'</code>	<code>%</code> both sides
Starts with B , ends with a	<code>LIKE 'B%a'</code>	combine "starts with" and "ends with"
Second letter is a	<code>LIKE '_a%'</code>	<code>_</code> is one unknown character
Starts with A , at least 3 characters	<code>LIKE 'A__%'</code>	combine wildcards
Exactly 4 characters	<code>LIKE '____'</code>	four underscores
Exact match	<code>LIKE 'Asha'</code>	with no wildcard the phrase must match exactly (same as <code>=</code>)

Try them:

```
SELECT emp_name FROM emp WHERE emp_name LIKE 'A%';
```

Result

emp_name
Asha

```
SELECT emp_name FROM emp WHERE emp_name LIKE '%a' ORDER BY emp_name;
```

Result

emp_name
Asha
Bala

emp_name
Chitra
Esha
Gita
Isha

```
SELECT emp_name FROM emp WHERE emp_name LIKE '%ha%' ORDER BY emp_name;
```

Result

emp_name
Asha
Esha
Farhan
Isha

```
SELECT emp_name FROM emp WHERE emp_name LIKE '_a%' ORDER BY emp_name;
```

Result

emp_name
Bala
Farhan
Hari
Jai

You can chain conditions with `AND` / `OR` — "starts with `A` or starts with `E`":

```
SELECT emp_name FROM emp WHERE emp_name LIKE 'A%' OR emp_name LIKE 'E%' ORDER BY emp_name;
```

Result

emp_name
Asha
Esha

Wildcards also work on other text: `WHERE City LIKE 'L_nd__'` matches *London* and *Lander* (`L`, any, `n`, `d`, any, any); `WHERE City LIKE '%L%'` matches any city containing an `L`.

The full wildcard family — and what Oracle supports

Symbol	Description	Oracle <code>LIKE</code>
<code>%</code>	zero or more characters	✓

Symbol	Description	Oracle LIKE
<code>_</code>	a single character	✓
<code>[]</code>	any single character within the brackets (<code>[bbsp]</code> = starts with b, s or p)	✗ SQL Server / MS Access only
<code>^</code>	any character not in the brackets (<code>[^bbsp]</code>)	✗ SQL Server / Access only
<code>-</code>	any single character within a range (<code>[a-f]</code>)	✗ SQL Server / Access only
<code>{ }</code>	an escaped character	(Oracle uses the <code>ESCAPE</code> clause)

Correction to a common note. W3Schools footnotes the bracket wildcards as "not supported in PostgreSQL and MySQL". They are actually **SQL Server / Access** features — **Oracle doesn't support them either**. `LIKE 'AR]'` in Oracle looks for a literal `]`. Rewrite it:

```
sql -- SQL Server: WHERE emp_name LIKE 'AR]' -- Oracle/MySQL: WHERE (emp_name LIKE 'A%' OR
emp_name LIKE 'R%') -- Oracle regex: WHERE REGEXP_LIKE(emp_name, '^AR]')
```

Matching a literal `%` or `_` — declare an escape character:

```
SELECT * FROM discounts WHERE label LIKE '50\%%' ESCAPE '\'; -- text that starts with "50%"
```

Case: Oracle's `LIKE` is case-sensitive (`'a%'` does not match `Asha`). Normalise both sides: `WHERE UPPER(emp_name) LIKE 'A%'` . (For pattern power beyond `LIKE` , see the regular-expression functions in Chapter 09.)

Performance note. `LIKE 'A%'` can use an index; `LIKE '%a'` and `LIKE '%ha%'` (leading wildcard) **cannot** — they scan the table.

4.7 Testing for NULL — `IS NULL` / `IS NOT NULL`

A NULL field is a field with **no value**. It is **different from zero and from spaces**. You cannot test it with `=`, `<` or `<>`; use `IS NULL` and `IS NOT NULL` .

```
SELECT column_names FROM table_name WHERE column_name IS NULL;
SELECT column_names FROM table_name WHERE column_name IS NOT NULL;
```

```
SELECT emp_name, comm FROM emp WHERE comm IS NOT NULL;
```

Result

emp_name	comm
Gita	5000
Hari	3000

```
SELECT emp_name, dept_id FROM emp WHERE dept_id IS NULL;
```

Result

emp_name	dept_id
Jai	NULL

4.8 ORDER BY — sorting the result

ORDER BY sorts the result set **ascending (default)** or **descending**.

```
SELECT column1, column2, ...  
FROM   table_name  
ORDER BY column1, column2, ... ASC | DESC;
```

```
SELECT emp_name, salary FROM emp ORDER BY salary;
```

Result

emp_name	salary
Isha	20000
Gita	35000
Hari	35000
Esha	40000
Jai	45000
Chitra	50000
Dinesh	50000
Farhan	65000
Bala	70000
Asha	90000

ASC is the default; DESC reverses it.

```
SELECT emp_name, salary FROM emp ORDER BY salary DESC, emp_name;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Farhan	65000
Chitra	50000
Dinesh	50000
Jai	45000

emp_name	salary
Esha	40000
Gita	35000
Hari	35000
Isha	20000

Sorting by several columns

`ORDER BY Country ASC, CustomerName DESC` sorts in **two levels**: first by `Country` ascending; **within each country**, by `CustomerName` descending. If the two directions are opposite that is perfectly fine — the second only applies *inside* each group of equal first values.

Source table:

customers

customer_name	country
Alice	USA
Bob	India
Charlie	USA
David	India
Eva	USA

```
SELECT * FROM customers ORDER BY country ASC, customer_name DESC;
```

Result

customer_name	country
David	India
Bob	India
Eva	USA
Charlie	USA
Alice	USA

Sorted first by `country` (ASC): India, USA. Then within each country by `customer_name` (DESC).

Other forms

You write	Meaning
<code>ORDER BY 2 DESC</code>	by the 2nd selected column (position) — works, but fragile
<code>ORDER BY annual_pay</code>	by a column alias — allowed here (ORDER BY runs after SELECT)
<code>ORDER BY salary * 12</code>	by an expression
<code>ORDER BY comm NULLS FIRST</code>	put NULLs first/last (default in Oracle: NULLs are the <i>largest</i> value — last for <code>ASC</code> , first for <code>DESC</code>)

You write	Meaning
<code>ORDER BY (CASE ...)</code>	by a computed rule — see §4.9

```
SELECT emp_name, comm FROM emp ORDER BY comm NULLS FIRST, emp_name;
```

Result

emp_name	comm
Asha	NULL
Bala	NULL
Chitra	NULL
Dinesh	NULL
Esha	NULL
Farhan	NULL
Isha	NULL
Jai	NULL
Hari	3000
Gita	5000

ORDER BY is the only way to guarantee order. Without it the database may return rows in any order, and that order can change tomorrow.

4.9 CASE — if / then / else inside SQL

The `CASE` expression goes through conditions and **returns a value when the first condition is met** (like an if-then-else). Once a condition is true it **stops reading** and returns the result. If no condition is true it returns the value in `ELSE`; if there is **no ELSE** and nothing matches, it returns `NULL`.

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  WHEN conditionN THEN resultN
  ELSE result
END;
```

(This is the "searched" form. The "simple" form compares one expression with values: `CASE job WHEN 'CEO' THEN 1 WHEN 'Manager' THEN 2 ELSE 3 END`.)

```

SELECT emp_name, salary,
       CASE
         WHEN salary > 60000 THEN 'High'
         WHEN salary BETWEEN 40000 AND 60000 THEN 'Medium'
         ELSE 'Low'
       END AS salary_band
FROM emp
ORDER BY salary DESC;

```

Result

emp_name	salary	salary_band
Asha	90000	High
Bala	70000	High
Farhan	65000	High
Chitra	50000	Medium
Dinesh	50000	Medium
Jai	45000	Medium
Esha	40000	Medium
Gita	35000	Low
Hari	35000	Low
Isha	20000	Low

Order matters — a row of 70,000 satisfies `> 60000` first and never reaches later branches.

CASE inside ORDER BY — "order the customers by City, but if City is NULL order by Country":

```

SELECT CustomerName, City, Country
FROM Customers
ORDER BY (CASE WHEN City IS NULL THEN Country ELSE City END);

```

Custom sort order — CEO first, then managers, then everyone else:

```

SELECT emp_name, job FROM emp
ORDER BY CASE job WHEN 'CEO' THEN 1 WHEN 'Manager' THEN 2 ELSE 3 END, emp_name;

```

Result

emp_name	job
Asha	CEO
Bala	Manager
Farhan	Manager
Chitra	Developer
Dinesh	Developer

emp_name	job
Esha	Tester
Gita	Sales Rep
Hari	Sales Rep
Isha	Intern
Jai	Consultant

`CASE` also powers **conditional aggregation** (`SUM(CASE WHEN ... THEN 1 ELSE 0 END)` , Chapter 06).

Oracle-only relative: `DECODE` . `DECODE(job, 'CEO', 1, 'Manager', 2, 3)` is the older Oracle shorthand for a simple `CASE` . It treats two NULLs as equal, which `CASE` does not. Prefer `CASE` — it is standard SQL and more readable.

4.10 Limiting rows — top-N queries

Oracle 12c and later: `FETCH FIRST`

```
SELECT column_name(s)
FROM   table_name
ORDER  BY column_name(s)
FETCH  FIRST n ROWS ONLY;
```

It returns the **first *n* rows *after* ordering** (ANSI-standard; introduced in Oracle 12c). `ORDER BY` is syntactically optional, but **without it "first" is not deterministic** — always pair them.

```
SELECT emp_name, salary FROM emp ORDER BY salary DESC FETCH FIRST 3 ROWS ONLY;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Farhan	65000

Variations:

Need	Syntax
Top <i>n</i>	<code>FETCH FIRST 3 ROWS ONLY</code>
Top <i>n</i> percent (sampling)	<code>FETCH FIRST 50 PERCENT ROWS ONLY</code>
Include ties for last place	<code>FETCH FIRST 3 ROWS WITH TIES</code> (needs <code>ORDER BY</code>)
Pagination — skip then take	<code>OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY</code>

```
SELECT emp_name, salary FROM emp ORDER BY salary DESC, emp_name
OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY;
```

Result

emp_name	salary
Chitra	50000
Dinesh	50000
Jai	45000

That is "page 2" with a page size of 3 (rows 4–6).

`WITH TIES` matters here: Chitra and Dinesh both earn 50,000. `FETCH FIRST 4 ROWS ONLY` would arbitrarily cut one of them; `FETCH FIRST 4 ROWS WITH TIES` returns both (5 rows).

Before Oracle 12c: `ROWNUM`

```
SELECT * FROM Customers WHERE ROWNUM <= 3;
```

`ROWNUM` numbers the rows **as they are fetched — before `ORDER BY` is applied**. So this is **wrong** for "top 3 earners":

```
SELECT emp_name, salary FROM emp WHERE ROWNUM <= 3 ORDER BY salary DESC;  -- X takes ANY 3 rows,
THEN sorts them
```

Say the first three stored rows are Asha, Bala and Chitra. The query takes those three and sorts them — the real third-highest earner, **Farhan (65,000)**, is never even looked at:

```
SELECT emp_name, salary FROM emp WHERE ROWNUM <= 3 ORDER BY salary DESC;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Chitra	50000

The fix is to **sort in a subquery first, then apply `ROWNUM` outside**:

```
SELECT * FROM (SELECT emp_name, salary FROM emp ORDER BY salary DESC)
WHERE ROWNUM <= 3;
```

Result

emp_name	salary
Asha	90000
Bala	70000

emp_name	salary
Farhan	65000

`ROWNUM` is assigned to a row only *after* it passes the `WHERE`, so `ROWNUM = 2` or `ROWNUM > 1` **never match anything** (the first candidate row always gets number 1 and fails).

The same idea in other databases

Database	Top 3
Oracle 12c+	<code>... ORDER BY x DESC FETCH FIRST 3 ROWS ONLY</code>
Oracle (old)	<code>SELECT * FROM (SELECT ... ORDER BY x DESC) WHERE ROWNUM <= 3</code>
MySQL, PostgreSQL, SQLite	<code>... ORDER BY x DESC LIMIT 3</code> (with <code>OFFSET n</code>)
SQL Server	<code>SELECT TOP 3 ... ORDER BY x DESC</code> or <code>OFFSET ... FETCH</code>

4.11 Aliases — temporary names

An **alias** is a **temporary name** for a column or a table. It renames output columns, shortens table names (especially in joins) and makes queries readable. **Aliases exist only while the query runs** — they never change the real names in the database.

Column aliases

```
SELECT column_name AS alias_name FROM table_name;
```

```
SELECT emp_name AS employee_name, salary AS monthly_salary FROM emp WHERE emp_id IN (101, 102);
```

Result

employee_name	monthly_salary
Asha	90000
Bala	70000

`AS` is optional: `SELECT emp_name employee_name, salary monthly_salary FROM emp;`

If the alias contains **spaces** (or must keep lower/upper case) put it in **double quotes**: `salary AS "Monthly Salary"`.

Aliases on expressions name calculated fields:

```
SELECT emp_name, salary * 12 AS annual_salary FROM emp WHERE emp_id IN (101, 102);
```

Result

emp_name	annual_salary
Asha	1080000
Bala	840000

Table aliases

```
SELECT t.column1, t.column2 FROM table_name t;
```

```
SELECT e.emp_name, d.dept_name
FROM   emp e
JOIN   dept d ON e.dept_id = d.dept_id;
```

-- without aliases: much longer and harder to repeat

```
SELECT emp.emp_name, dept.dept_name
FROM   emp
JOIN   dept ON emp.dept_id = dept.dept_id;
```

Oracle vs others — no AS before a table alias. ANSI SQL, MySQL and PostgreSQL allow `FROM emp AS e`.

Oracle does not — it raises `ORA-00933: SQL command not properly ended`. In Oracle write `FROM emp e`. (`AS` is fine for *column* aliases in every database.) Many tutorials that use `Employees AS e` will not run unchanged on Oracle.

A **join alias is a must-use** when the same table appears twice (a self join, Chapter 07):

```
SELECT e.emp_name, m.emp_name AS manager
FROM   emp e
JOIN   emp m ON e.manager_id = m.emp_id;
```

Subquery aliases name a temporary result: `FROM (SELECT emp_name, salary FROM emp WHERE salary > 50000) high_earners` — here `high_earners` acts as a temporary table (a *derived table*/inline view).

Alias type	Use	Example
Column alias	rename an output column	<code>salary AS monthly_salary</code>
Table alias	shorten a table reference	<code>emp e</code>
Expression	name a calculated field	<code>price * qty AS total_price</code>
Subquery	name a temporary result set	<code>(SELECT ...) temp_table</code>

Trap — aliases can't be used in WHERE. `SELECT salary*12 AS annual FROM emp WHERE annual > 500000` fails (`ORA-00904: invalid identifier`) because `WHERE` runs before `SELECT`. Repeat the expression, or wrap the query: `SELECT * FROM (SELECT salary*12 AS annual FROM emp) WHERE annual > 500000`.

Interview corner

Q. In what order does SQL execute a query? `FROM`/`JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `FETCH/ROWNUM`.

Q. Why can't I use a column alias in WHERE? `WHERE` runs before `SELECT` creates the alias. (You *can* use it in `ORDER BY`.)

Q. What is the precedence of AND, OR, NOT? NOT, then AND, then OR. Use parentheses for mixed conditions.

Q. Is BETWEEN inclusive? Yes — both ends are included; the smaller value must come first.

Q. Difference between % and _ in LIKE? % = any number of characters (including none); _ = exactly one.

Q. Find names that start with A or R in Oracle. WHERE emp_name LIKE 'A%' OR emp_name LIKE 'R%' (or REGEXP_LIKE(emp_name, '^[AR]')). The [AR]% form only works in SQL Server.

Q. How do you get the top 3 salaries in Oracle? 12c+: ORDER BY salary DESC FETCH FIRST 3 ROWS ONLY. Older: sort in a subquery, then WHERE ROWNUM <= 3 outside it — never put ORDER BY and ROWNUM in the same query level.

Q. Why does WHERE ROWNUM = 2 return nothing? ROWNUM is assigned as rows are accepted; the first row is numbered 1, fails = 2, and the counter never advances.

Q. NOT IN vs <>? NOT IN (a, b) = <> a AND <> b. Both drop NULLs.

Q. SELECT DISTINCT vs GROUP BY? Both remove duplicates; GROUP BY additionally lets you aggregate per group. For plain de-duplication use DISTINCT.

Q. Can you use AS for table aliases in Oracle? No — write FROM emp e.

Quick recap

- SELECT cols FROM t WHERE ... ORDER BY ...; **execution order:** FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → FETCH.
- Text in 'single quotes', numbers bare; Oracle text comparison is **case-sensitive**.
- NOT > AND > OR; parenthesise. IN = many OR s; BETWEEN is **inclusive**; LIKE: % many, _ one; [] wildcards are **not** Oracle.
- IS NULL / IS NOT NULL, never = NULL.
- ORDER BY ASC (default) / DESC, multi-column, NULLS FIRST/LAST.
- CASE WHEN ... THEN ... ELSE ... END — no ELSE ⇒ NULL.
- Top-N: FETCH FIRST n ROWS ONLY (12c+) or sort-then-ROWNUM in a subquery.
- Aliases: AS optional for columns; **never AS for Oracle table aliases**; not usable in WHERE.